

Final Project Report: Social Tasker

Bachelor of Science in Computer Science - University of London

By: Bernard Oosthuizen

Table of Contents

Table of Contents	2
Introduction	3
Literary Review	5
Gamification	5
Existing Solutions	7
<i>Asana</i>	7
<i>Todoist</i>	8
Graph Databases	9
Rewards System	10
App Design	13
Architecture	19
Project Timeline	22
Implementation	26
Features	26
Rewards System	27
Code Review	30
The Interface	38
Evaluation	39
Testing	40
Critical Evaluation	45
Conclusion	46
Links	48
References	49

Introduction

Being part of a community is an integral part of our lives. With the dawn of the work-from-home initiative, having a sense of community can feel challenging. This project aims to promote social connections while users do their daily tasks. The Mobile Development: Task Manager Mobile App template was used for this project. Social Tasker is an iOS and Android application that helps users track and organise tasks. The application will allow users to create tasks. Priority, a due date and a description can be added to each task. Users can arrange their tasks based on due date or priority. The key innovation of Social Tasker is how it fosters a sense of community and social engagement around task completion. Users can invite others into their community, and the application encourages users to support and compete with each other. Users earn points when completing tasks, with the point values determined by factors like task priority, duration, creation date, and the user's consistency in completing tasks. By linking task management to social interaction and accountability, Social Tasker aims to help distributed workers and solopreneurs maintain a stronger sense of community, even as they work independently. The focus on consistency encourages the development of good productivity habits, which improves user retention over time.

Studies have shown that the gamification of mobile applications increases user retention and can make them more productive. When a user's intrinsic motivation to use a platform is complemented by extrinsic motivational factors such as rewards or social recognition, significantly higher levels of user engagement are present and sustained usage over time [4, 12]. Gamified applications incorporate challenges or rewards in activities where they are not usually present. Features like rewards, challenges and leaderboards are included in gamified applications [4]. By incorporating these motivational factors into otherwise mundane tasks, gamified applications transform the user experience. Completing a task suddenly becomes intrinsically rewarding, as users work to earn points, climb leaderboards, and obtain virtual rewards. Existing task management applications have a heavy focus on team

collaboration. Their users are mostly individuals within the same organisation or project. Essential features are common amongst competitors and include; due dates, task priority and task descriptions to name a few [2, 5]. With this apparent gap in the market, Social Tasker aims to create a sense of friendly competition and social accountability for the solopreneur or independent workers whilst fostering a sense of shared purpose and community.

Social Tasker will include some novel technology to address unique challenges within a social network-based platform. Structuring a social network is challenging in conventional relational databases. The complex connections between users, tasks, and social interactions require a data model that can efficiently capture these relationships. Querying relationships can be very computationally expensive. To overcome these challenges graph databases are used in favour of traditional relational-based systems. These are ideal for social networks and make creating and querying connections efficient and easy [7]. In addition to the underlying data architecture, Social Tasker will also require the development of a custom rewards algorithm to power the gamification elements. This algorithm will need to intelligently calculate point allocations based on a variety of factors, including task priority, due dates, creation timestamps, and the user's consistency in completing their tasks.

This report outlines the feasibility of Social Tasker based on the past success of gamified applications across various industries. The success of gamification serves as a good foundation for the Social Tasker concept. Through the analysis of current task management platforms common features and limitations are identified. It becomes apparent that creating a sense of community is a key area where Social Tasker differentiates itself. With this comprehensive understanding of the problem space and existing landscape, the report then outlines the detailed design and technical architecture of the application. A clear development and testing plan is also laid out to ensure a timely and robust product delivery. This roadmap highlights key milestones and quality assurance measures to mitigate any unforeseen circumstances.

The objective of Social Tasker is to inspire people from all walks of life to be more productive. Increased productivity enables users to do the things important to them. Spending more time with loved ones, working on that hobby or taking time to oneself not only improves mental health but also serves as recovery to perform better where it matters.

Literary Review

Gamification

Integrating game elements into applications is referred to as gamification. The goal of gamification is to transfer some of the positive characteristics of games to something that is not a game. By incorporating gamification elements such as points, badges, leaderboards, and challenges, developers aim to enhance user engagement, motivation, and behaviour change within applications.

Studies have revealed that gamification can enhance user engagement and productivity. However, the long-term effectiveness of such an application relies heavily on the alignment of the application's design principle with the user's intrinsic motivation [4]. Intrinsic motivation refers to the satisfaction or interest in an activity without any external reward. In the context of a task manager application, this may translate into the joy of completing tasks on time and efficiently. Extrinsic motivation on the other hand is driven by external feedback such as rewards or social recognition [12]. A study by Kappen et al. [2013] found that gamification of task management applications does increase productivity and engagement. This study gave participants a heavily gamified application to use. The application had users create characters and complete quests to receive rewards. Participants confirmed that although the test application's reward system was exciting at first (extrinsic motivations), their extended use of the application was motivated by the necessity to efficiently complete tasks (intrinsic motivation). In conclusion, the study determined that their heavily gamified application incorporated too many gamified elements. The gaming aspect took away from the

application's goal of productivity [4]. A careful equilibrium between playfulness and productivity is essential in increasing engagement and maintaining the application's productivity aspect.

The health and fitness genre of applications has seen great success in the use of gamification. These applications incorporate challenges, acknowledgements, guidance and social experiences to foster user engagement. Fitness applications have been successful in part due to the intrinsic motivation of their users to keep moving in some shape or form [4]. The physical and psychological benefits of exercise are well documented. It is not surprising that users are intrinsically motivated to exercise and use fitness apps in the long term. A study by Rehman et al. [2024] suggests that connecting with like-minded plays a role in user engagement in gamified applications. This study discusses the intricacies of crafting an experience that incorporates features that resonate with the user's needs whilst maintaining engagement. The same study suggests a blend of gamified elements, social sharing and collaborative challenges can stimulate an individual's intrinsic motivation [12].

These studies provide insight into the gamification of applications and help in determining how to 'gamify' a task management application. The studies did not include a large test pool, potentially skewing results. Even though the participants were not as diverse as one would hope, these studies pointed out some key elements of gamification based on previous literature and studies. 1. It is difficult to change a user's intrinsic motivators but can be influenced by their peers and environment. 2. Too much gamification harms the user experience and productivity. A task management application is meant to be easy to use and should not take away from the user's productivity.

Gamification also has some ethical considerations. Gamification is widely used in social media to keep users engaged on a platform. Social media giants have been criticised for this technique. Although it is not an explicit dark design pattern, gamification, much like gambling, does psychologically manipulate a user to use a platform. This manipulation can lead to excessive or addictive use, which may have negative impacts on mental health and productivity. There are also concerns that gamification can exploit cognitive biases and be

used to nudge users towards behaviors that benefit the platform operator rather than the user. Careful design is required to ensure that gamification enhances the user experience in an ethical manner, rather than prioritizing engagement metrics over user wellbeing. Developers should consider the potential downsides and unintended consequences of introducing gamification elements, and strive to implement them in a way that is transparent, empowering, and aligned with the users' best interests.

Based on the above literature, gamified applications should include a reward system, a community aspect like a leaderboard and challenges to maintain user engagement [4]. The literature also affirms the idea of a social-based task management application. Users using 'Social Tasker' will already be intrinsically motivated to be productive. Connecting these users with like-minded individuals will enhance their productivity and engagement in their daily tasks and on the application.

Existing Solutions

Several products in this space help individuals and teams organise, track and collaborate on their work. Two popular products dominate the task management space and provide a good research foundation to explore features and user experiences enjoyed by many. Asana is a feature-rich task management app designed for team collaboration and project management, while Todoist focuses on simplicity and individual productivity. This section highlights the key features and capabilities of these applications to establish which are expected from users. These learnings will be used to determine the minimum features and design elements needed for Social Tasker.

Asana

Asana helps teams and individuals manage their tasks. It has a project management focus and provides a comprehensive set of features to enhance productivity and organisation within a team. Asana's intuitive interface allows users to create tasks, set due dates, assign

responsibilities, and track progress in a visually appealing and user-friendly manner. Users can view tasks in list-, board- or calendar-style views. Collaboration is an important aspect of Asana, it provides users with the ability to create projects, add team members, assign tasks, and communicate within the platform. Commenting and tagging features enable seamless communication, ensuring that team members can easily discuss tasks, share updates, and provide feedback, all within the context of the relevant tasks. Users can attach files to tasks to create more context for the given task. Several dashboards and reports give an overview of project progress [2]. Asana's design is minimalist with a minimal use of colour to not distract from its fundamental purpose.

Todoist

Todoist focuses on simplicity and individual productivity. Its intuitive design makes it quick and easy for users to create tasks, set due dates, and organise them into projects and subprojects. Users are also able to prioritise and view tasks in their preferred style. Todoist's 'Karma' feature motivates users to complete tasks through a reward system. Users set goals each day and receive points based on how many of these tasks are completed. The reward system also takes into account your daily and weekly completion streak. The more consistent the user is the more points they earn [5]. Similar to Asana, Todoist's design is also minimalist and uses colours to communicate to the user rather than just for design.

These platforms each have their strengths; Asana prioritises collaboration and Todoist focuses on simplicity for the individual user. Features such as due dates, priorities, status, and different ways of arranging tasks and commenting are the most fundamental for a task management application based on these examples and literary research [2, 4, 5]. Social Tasker will have to include all of these basic features in its minimum viable product (MVP) stage. Inspired by Todoist, Social Tasker should have a simple and easy-to-use interface so that individuals can navigate through the application intuitively. Similar to Todoist's reward system, consistency as well as task completion should be rewarded. Requiring consistent

use of the application to gain rewards will enhance user engagement. A mechanism to avoid cheating has to be implemented too.

Graph Databases

The premise of Social Tasker is a community that users are a part of, inspiring each other to be more productive. This community is a social network connecting users to one another. Connection between entities can be represented through graphs. Graphs consist of nodes and edges, through which entities and relationships can be represented. A node of the graph would represent a person whilst its edges represent relationships between nodes or in our case people. Soad Almadby [2018] explores the benefits of using graph databases to store data with inferred relationships. The power of a graph database lies in its ability to easily and efficiently query connected data. Relational databases are less efficient in this sense, with studies showing a significantly longer retrieval time with multi-layered relationships compared to that of a graph database such as Neo4j. Neo4j is an open-source graph database database management system (DBMS) that enables developers to easily create graph databases. It provides simple create, read, update and delete (CRUD) operations that would otherwise be incredibly complex in traditional databases. Neo4J is ideal for implementing a social network.

A hybrid approach will be used for Social Tasker's database design. Inspired by Vyawahare et al. [2018] data will be stored based on its use instead of passing all data through a classifier as described in their research. Separating data and databases by concern has a couple of benefits; mainly enhanced performance, query only what is required most efficiently, and enhanced security [7]. Separating social connections and personal data enables developers to restrict access to personal data more efficiently. For instance, Social Tasker may store usernames, relationships and rewards in a graph database that is accessible to all users but store user information in a relational database where entries are

only accessible by the users whose details it is. Gaining access to the graph database would therefore not reveal any sensitive information.

Rewards System

Social Tasker's reward system will reward users for completing daily tasks and consistency over multiple days and weeks. Frazier-Roberts and Peoples [2019] published an algorithm for a gamified application that rewards students for practising musical instruments. Their algorithm gives good guidance on how to calculate a user's consistency and how to allocate points for completed tasks. The pseudocode in Figure 1 below describes a method of allocating points based on task / musical piece difficulty. A similar algorithm could be used in Social Tasker, substituting 'piece difficulty' with a blend of task creation date, due date and task priority. The resulting score would reward users for completing high-priority tasks on time. Using task duration also came to mind but could be flawed in some ways. Users can easily cheat by underreporting a task's estimated duration and initially, not enough data on users and their tasks is available to make a good estimation on behalf of the user.

Activity: Completing Pieces

Activity Data:

When a piece is marked as completed, new points can be added to the student's score.

Quality Standardisation and Criteria Construction:

The difficulty of the piece will be divided by the student's average grade. This is then multiplied by the piece's achievable score, and rounded.

Criteria Integration:

This score is added to the student's existing score.

```
Input: Piece P, Student S
Output: New Score NS
1: BEGIN
2: SET Difficulty D = Piece.difficulty
3: SET average A = Student.averageGrade
4: SET achievable Score ACS = Piece.score
5: LET Score NS = D / A
6: LET NS = NS * ACS
7: RETURN NS
8: END
```

Figure 1. Task completion algorithm [11]

Figure 2 describes a method for rewarding a user's consistency. It looks at the user's history and calculates whether their practice duration is similar to that of a given period and awards consistency badges if it is. For Social Tasker, the user could be asked to enter the duration of completed tasks and a similar algorithm implemented. Another method may look at the average points over a given time instead of duration. This will take into account missed deadlines and priorities.

Activity: Maintaining a Constant Practice History

Activity Data:

From the student's PracticeWeek list, aggregate all of the durations for each day by filtering the map given

Quality Standardisation and Criteria Construction:

Calculate the student's average practice duration for a pre-defined number of months. For each practice duration, check if it is within 20% of the average practice duration.

Criteria Integration:

If more than 80% of the practice durations are within 20% of the average practice duration, award the badge for maintaining a constant practice history.

```

Input: Student S
Output: amended Student S
1: BEGIN
2: LET DPA = new Array // array to hold durations of daily practice
3: LET VPA = new Array // array to hold validated practice durations
4: SET A = S.practiceWeeks
5: WHILE A HAS NEXT; DO
6: SET B = A.next
7:   WHILE B.practicedItem HAS NEXT;DO
8:     SET C = B.practicedItem.next
9:     WHILE C has NEXT; DO
10:      DPA ADD (C.duration)
11:    END WHILE
12:  END WHILE
13: END WHILE

```

Figure 2. Rewarding consistency [11]

A mechanism will be implemented to prevent users from only completing high-priority tasks. Weighting tasks based on their creation date, priority and due date may be a good way of handling this.

This literary review has explored the landscape of application gamification, user motivations, existing task management applications and the technology driving these products. The research shows that gamification increases user engagement and can act as a motivator. Social Tasker is a good addition to the task management space, as users will benefit from

combining features of products that they already use and adding a social aspect to bring them a sense of community missing in the current offerings.

The research findings summarized in Figure 3 provide a solid foundation for developing a robust, feature-rich version of Social Tasker that is grounded in a deep understanding of current market offerings and technological capabilities.

Research Area	Findings	Practical Applications
Gamification	- A sense of community enhances engagement. - Features include rewards, feedback and social recognition.	- Users can add friends and view each other's scores on a leaderboard. - Users receive reward points for consistently completing tasks.
Existing Solutions	Features: - Task creation - Set task due date - Set task priorities - Task status - Different viewing options	Social Tasker Features: - Task creation - Set task due date - Set task priorities - Task status - List view (by due date or priority) - Community Leaderboard - Task stats
Databases	- Graph database (Neo4j) best suited for social media apps. - Hybrid solutions have performance and security benefits.	- Use Neo4j to map relationships between users. Only include usernames and 'points' in this database. - Use a relational database to store personal information and restrict access to entries.
Reward System	- Earn points based on consistency. - Earn points for completion of tasks based on task difficulty.	- Users earn badges based on average task completion in a week. Increases point earning potential. - Earn points for tasks completed based on creation date, due date & completion date.

Figure 3. Social Tasker features based on Literature review

App Design

The overall application design for Social Tasker follows a minimalist approach, prioritising productivity and efficiency as the primary features. Numerous studies have shown that users are drawn to clean, uncluttered interfaces that streamline their workflows and minimise distractions [4, 5]. By adhering to a minimalist design philosophy, Social Tasker aims to create a highly intuitive and focused experience for its users. The gamification elements should be carefully balanced within this minimalist framework. Excessive or overbearing game mechanics can distract the user from the application's productivity features. Therefore, the gamification aspects of Social Tasker have been designed to be unobtrusive and complement the task management experience [4].

To realise this vision, a series of wireframes has been developed that illustrate the key screens and user flows within the application. Figure 4 provides extracts from these wireframes, giving a visual representation of the content and navigational structure of the Social Tasker interface. The wireframing process was carried out using Figma, a collaborative design tool that enables the creation of low-fidelity prototypes and clickable mockups [6]. This allows for quick iteration of the interface design and optimising the user experience before committing resources to high-fidelity development.

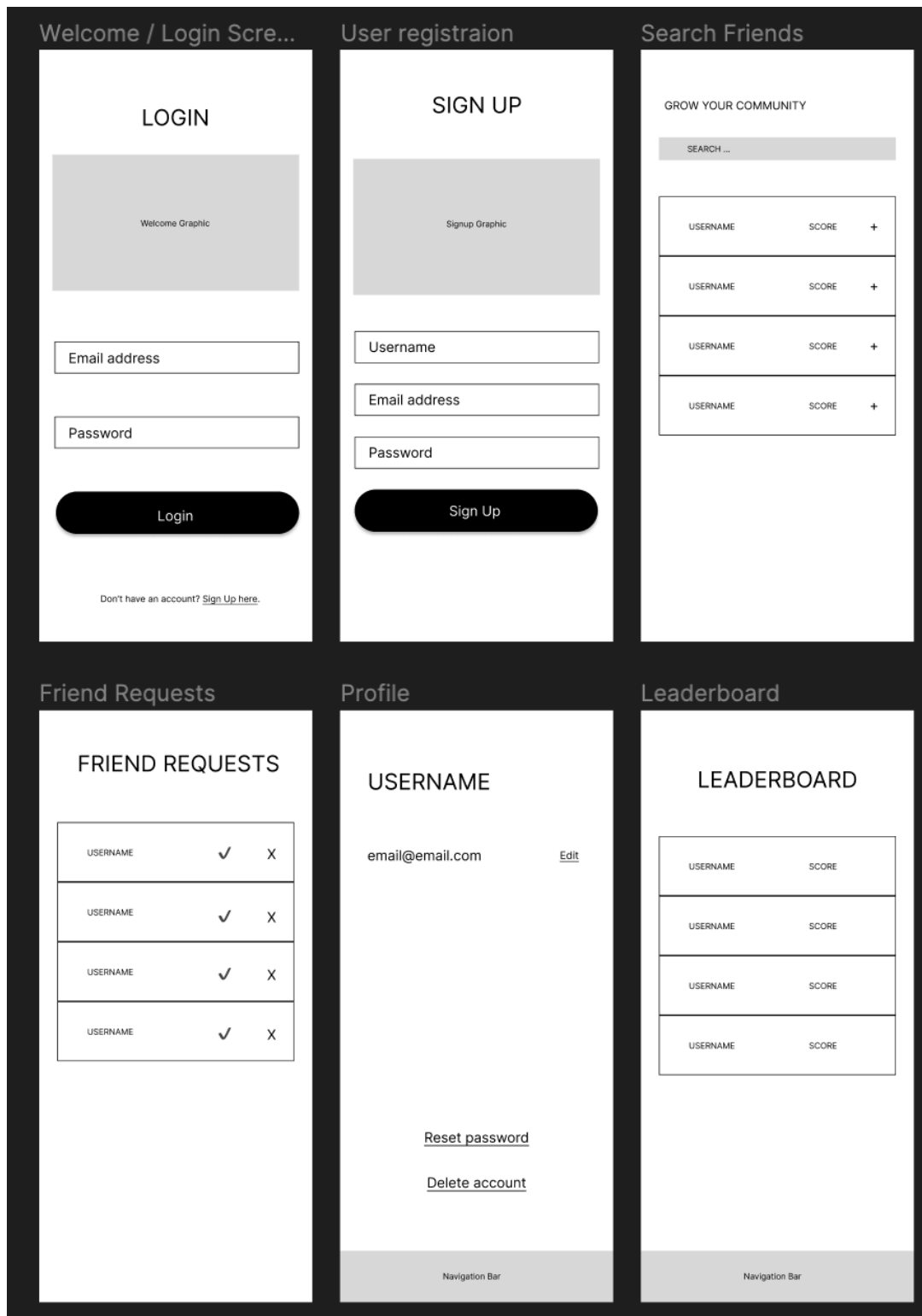


Figure 4: Wireframe

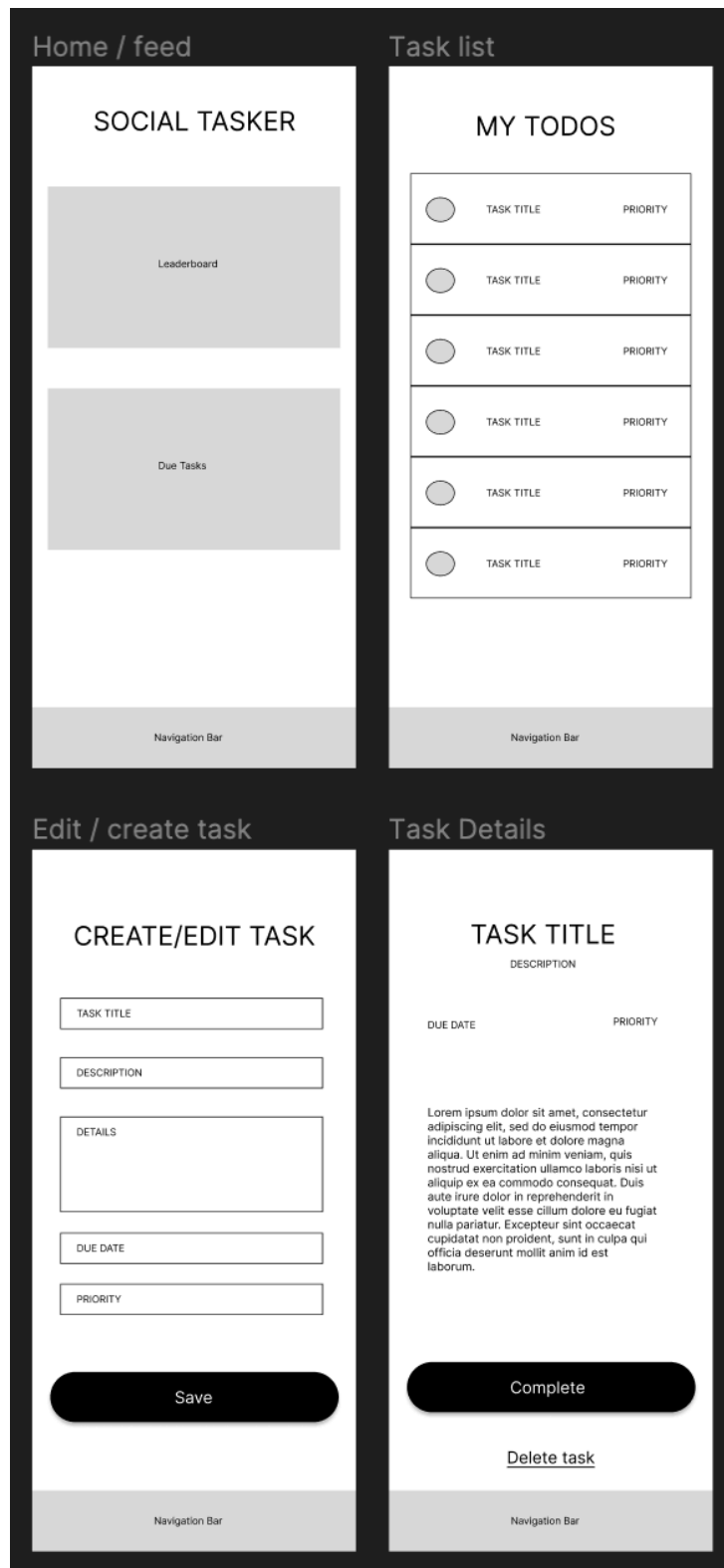


Figure 4: Wireframe

Building upon the wireframes, an advanced, high-fidelity prototype was also created using Figma, as shown in Figure 5. This prototype is very closely aligned with the end product's design. It has some basic functionality like page navigation which mimics some of the interaction between the user and the product. Having this interactive prototype available serves to enable early-stage user testing and feedback collection; ultimately making development quicker and more efficient.

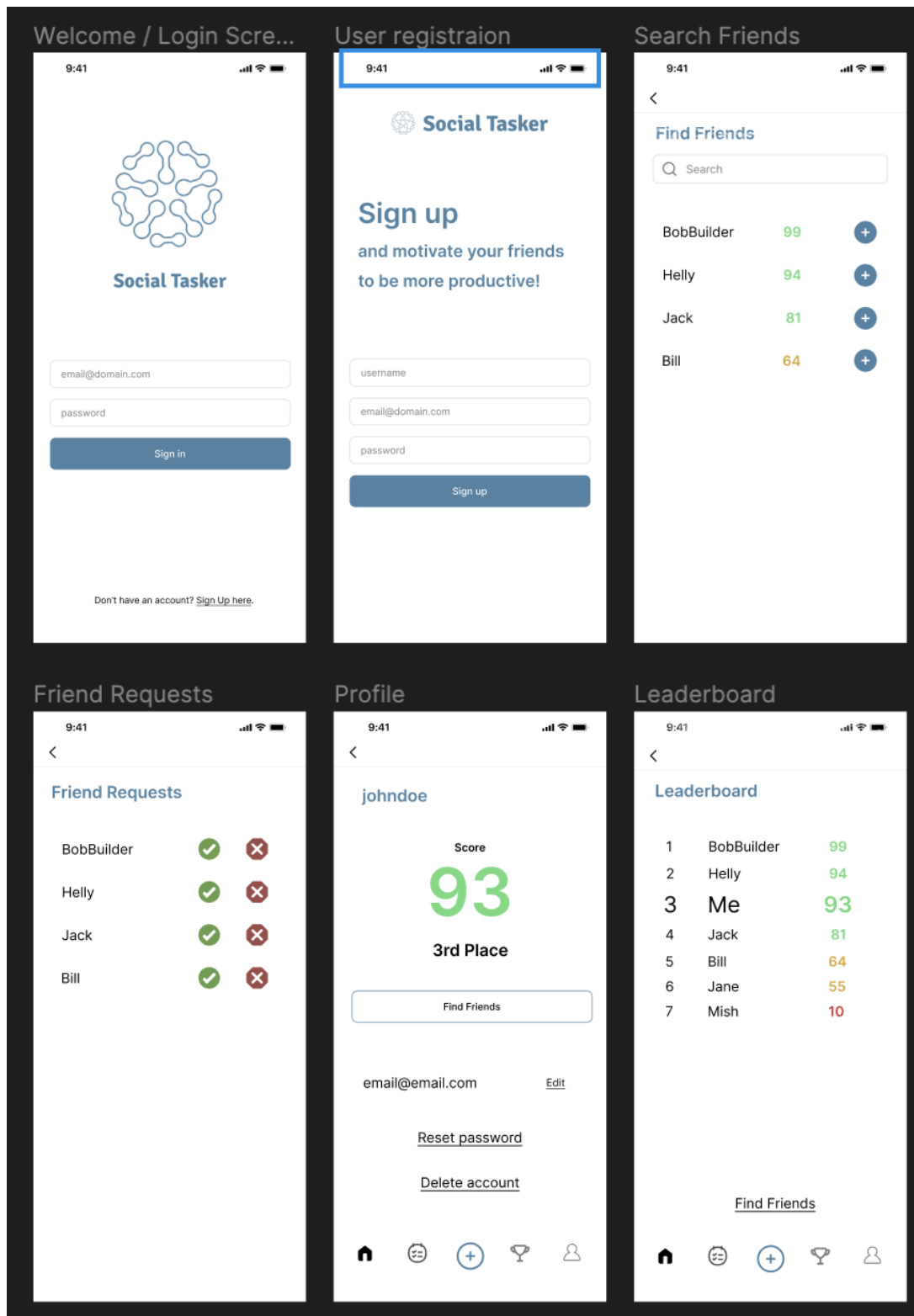


Figure 5: Prototype

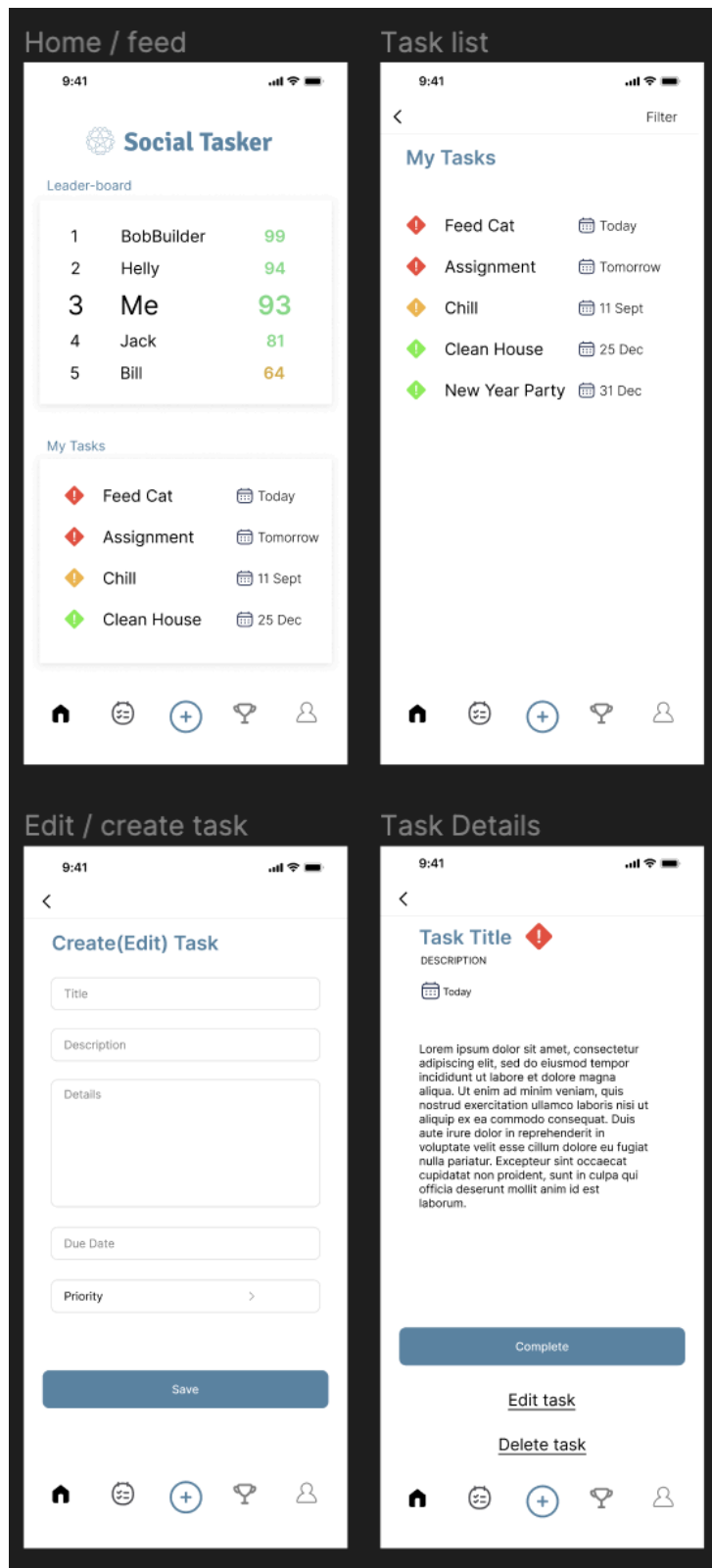


Figure 5: Prototype

Architecture

The application's infrastructure will be hosted on Google Firebase and is based on the principle of microservices [1]. Firebase provides tools for developers to easily get up and running with databases and back-end services for web and mobile applications. As illustrated in Figure 6, the Social Tasker application will have a front-end interface that connects to a set of backend services running on Firebase Functions, which are serverless cloud functions provided by the Firebase platform. This integration is facilitated through the use of Firebase's Software Development Kit (SDK), which abstracts away the underlying HTTP request-response mechanism and simplifies the development process. An API gateway with throttling and access control does not need set up and is taken care of by Firebase through the SDK. When a user takes an action that requires a database entry or modification, the SDK sends the request to the relevant microservice. These Firebase Functions contain the middleware code that executes queries on the database. Firebase provides triggers that fire when database modifications are made. A Firebase Function will execute when a change in a user's score is detected. It will then update the Neo4j graph database with the user's score. The choice to utilise a graph database, specifically Neo4j, for the application's data storage is a strategic one. Graph databases excel at modelling and querying complex relationships, which aligns well with the social network aspect of the Social Tasker application.

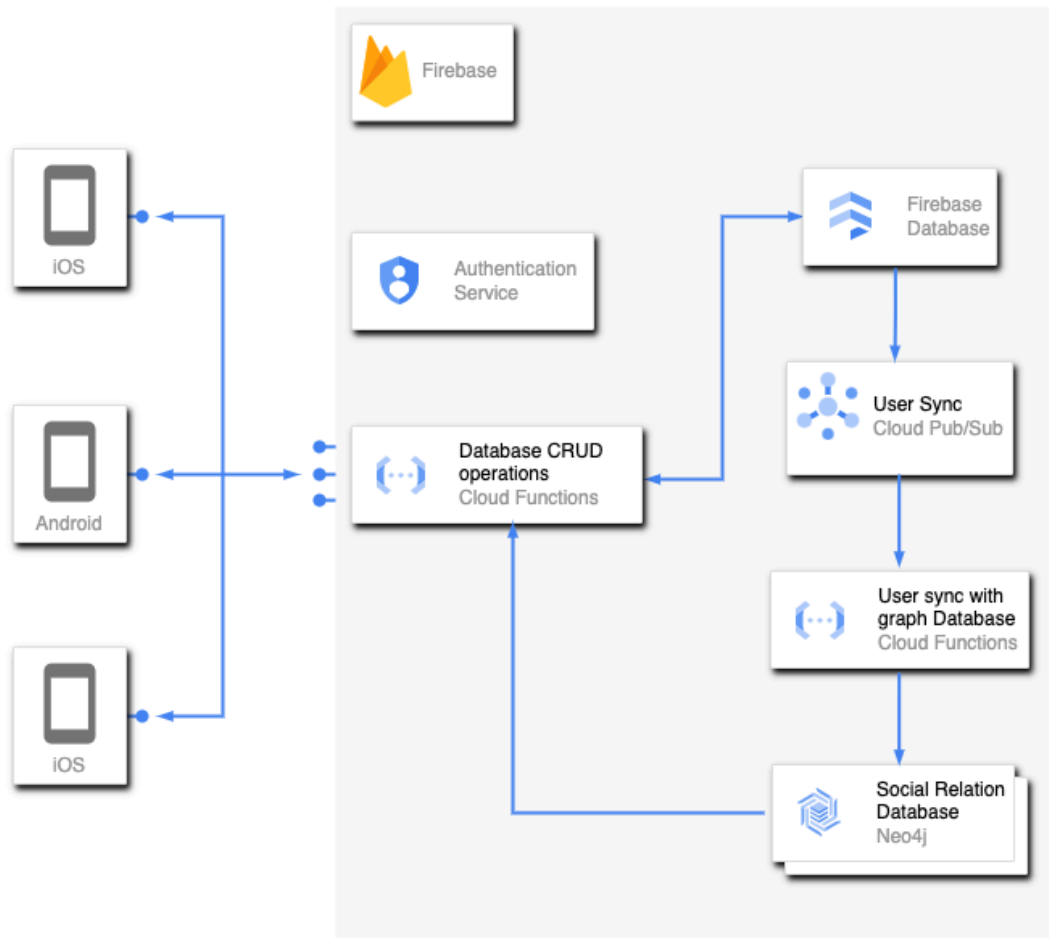


Figure 6: Architecture

Utilising a hybrid database design, the graph database will store the user's username, score and relationship to other users, whilst task and user information will be stored in a traditional relational system [9]. Keeping this information separate provides an extra layer of security. Stringent access control can be implemented in sensitive information in the user and task tables. This hybrid design was implemented through a REST API. A single endpoint for actions will take care of both database operations. This has the benefit that the frontend code is minimal, calling only one API endpoint, and that the synchronisation across

databases is taken care of by solely the backend. For instance, when a user is created the relevant API endpoint is called. The backend then takes care of creating the user in both databases. A challenge here is to roll back any changes made should errors occur in any of the database actions. This challenge was overcome by temporarily saving old data and rewriting it to the database in case of any errors, keeping the two databases in sync.

Figure 7 further expands on the databases' schemas. The user table will include personal information. Items like user_id and user_score will be synchronised with the graph database. Login credentials do not need to be saved here since authentication will be handled by Firebase Auth, an all-in-one authentication solution from Google Firebase. User passwords are automatically encrypted and stored securely following industry best practices. The task table includes all task-relevant information. A WebSocket will be established between the friendRequest table and the client. This will enable instant two-way communication between the client and the table allowing for near-instant delivery of friend request information.

Relational Database

user
user_id INT
user_email VARCHAR
user_name VARCHAR
user_score INT

task
task_id INT
user_id INT
task_title VARCHAR
task_content VARCHAR
task_due_date DATE
task_create_date DATE
task_priority INT
task_last_updated DATE

friendRequest
request_id INT
from_id INT
to_id INT
status VARCHAR

Graph Database

user
user_id INT
user_name VARCHAR
user_score INT

Figure 7: Database Schemas

Project Timeline

The development of Social Tasker is being executed using agile project management methodologies. Agile approaches, with their emphasis on iterative techniques and continuous improvement, are particularly well-suited for software projects where requirements and solutions can evolve based on user feedback and testing [3].

To structure the project the development has been divided into 5 key milestones: the midterm assignment, frontend development, backend development, testing, and the final report. Setting these milestones helps the team track progress, identify and address any issues, and maintain clear goals.

The frontend milestone includes all user-facing design and interaction elements. Once the frontend is completed, the backend authentication, database, and algorithms are developed and connected to the frontend. This phased approach allows developers to tackle the application in logical, manageable chunks. Figure 8 is a timeline view of the project's milestones/deliverables. Issues (tasks) are created in Linear and assigned a milestone, due date and status. An issue is created for each feature or function. For example, the login screen and the creation of tasks in the database are each an issue. Team members can view issues assigned to them in multiple different ways. Viewing issues according to milestones helps keep the focus on the current deadline. Issues are displayed on a Kanban-styled board sorted by their status. Issues are moved between columns based on

their

status.

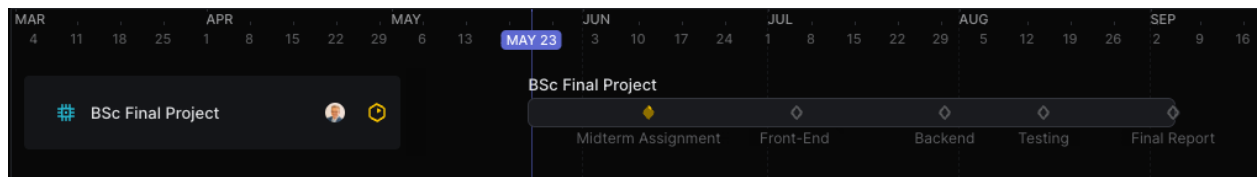


Figure 8: Project Timeline

Figures 9 and 10 show the issues created for the midterm and frontend milestones respectively. Linear also provides valuable project tracking and progress monitoring capabilities. Figure 11 shows a graph that gives the team an overview of the project's progress, with visual indicators of issues completed, in progress, and remaining. This data-driven insight helps the team identify potential bottlenecks, make informed decisions, and ensure that the project stays on track towards its goals [8].

By leveraging agile methodologies and robust project management tools, the Social Tasker team is well-equipped to deliver the application within the established timeline, while maintaining the flexibility to adapt to evolving requirements and user feedback.

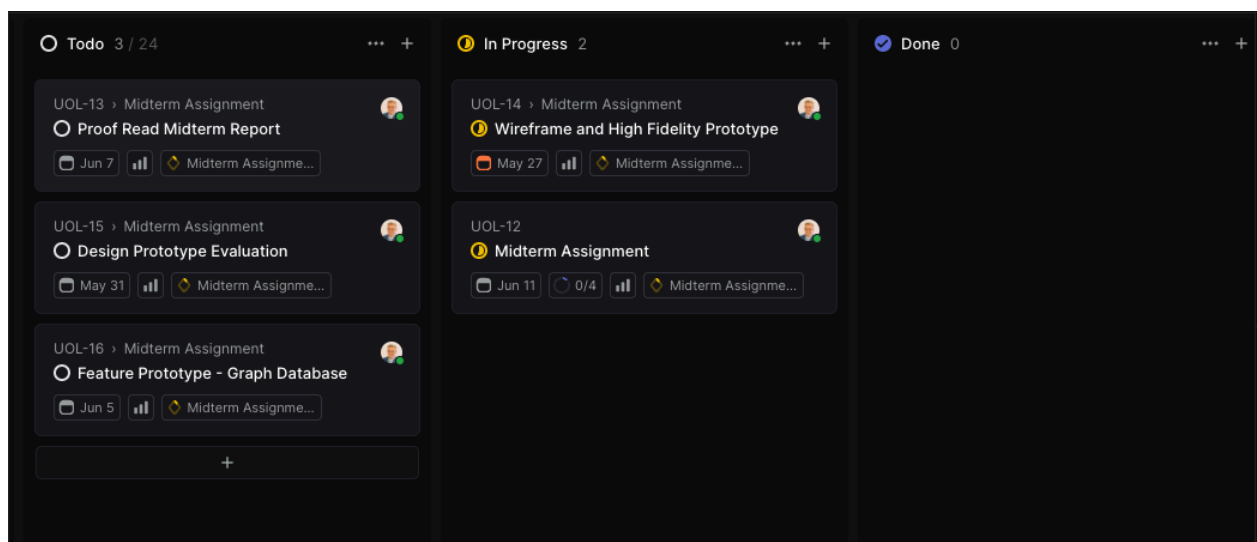


Figure 9: Midterm Milestone Issues

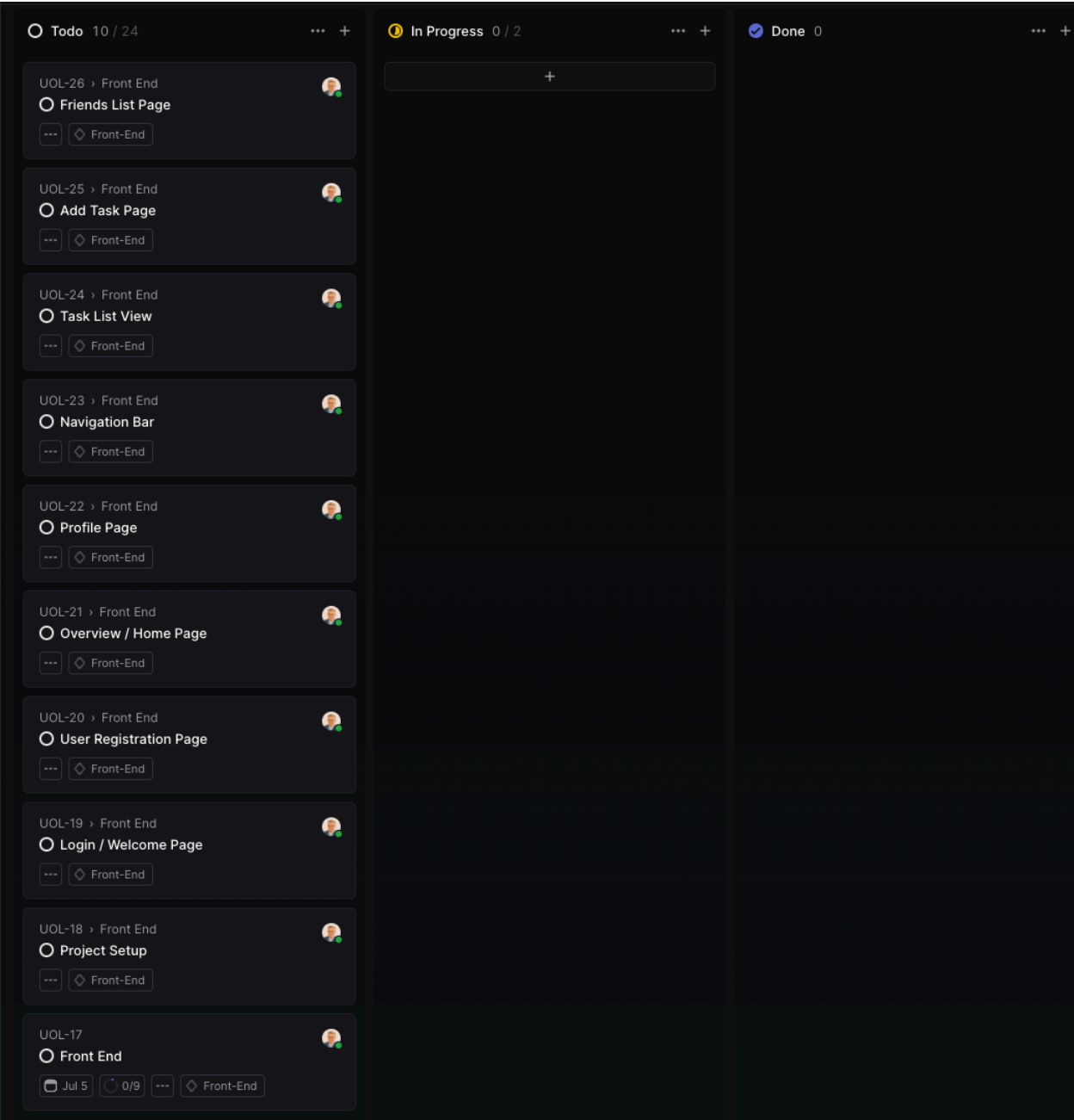


Figure 10: Frontend Milestone Issues

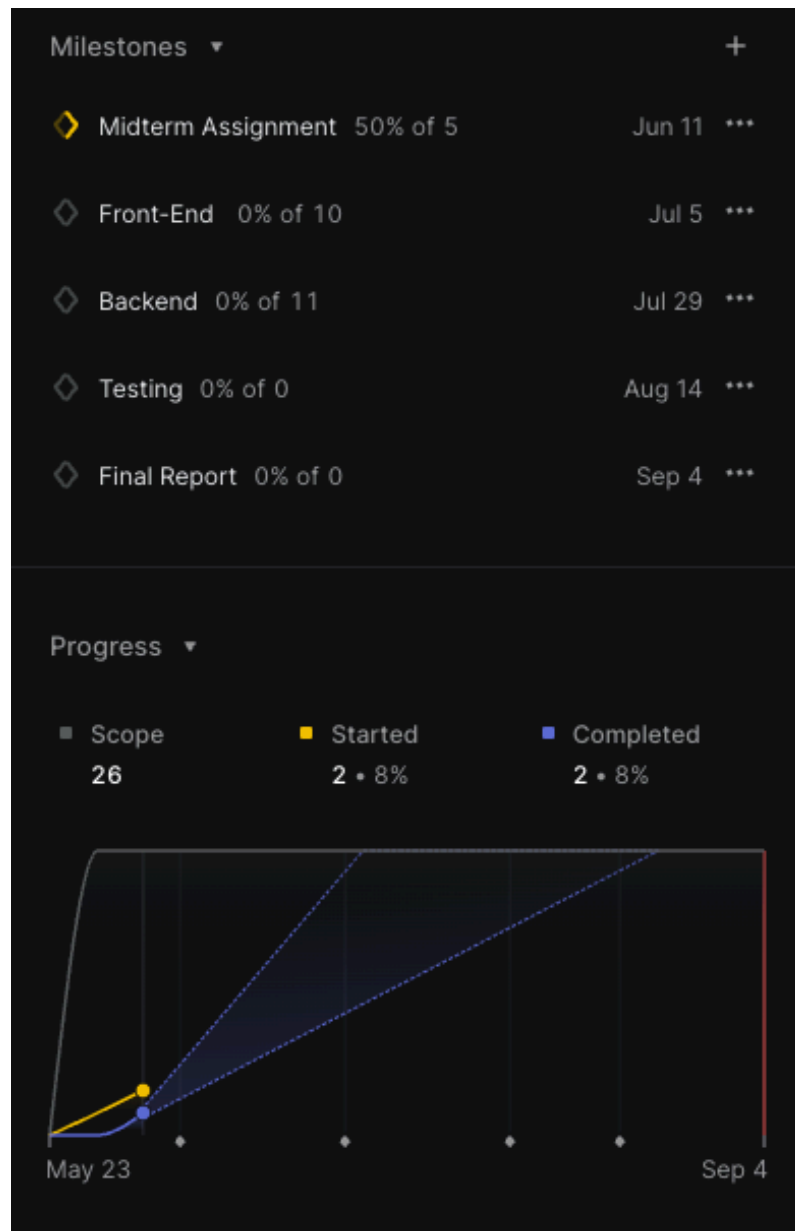


Figure 11: Project Progress Graph

Implementation

Features

Social Tasker encourages users to compete in completing tasks and assigns scores to users which are then viewable to their community on a leaderboard. The app allows users to navigate between pages through a navigation bar at the bottom of their screen. The navigation bar allows for quick navigation to the home/dashboard, task list, task creation, leaderboard and and profile pages. Users create tasks and assign a set of priorities to each task. Details such as the task due date, a short description and the expanded details of the task can also be added to each task. Upon saving a task the user can view their tasks on the dashboard of the app or in a dedicated view displaying all of the user's tasks. Herein tasks are displayed in a list view with their priority, title and due date displayed. The task list view allows the user to sort the tasks by due date in either ascending or descending order. Tasks can be edited after creation, enabling users to change any of its details.

Users are required to create an account to use Social Tasker. All tasks are synced to the cloud and are available across multiple devices. Users can view their details on a profile page where they are also able to delete their account, reset their password or log out of their account. Here, a defining feature of Social Tasker is also present, the user's score and ability to create a community. Their score is displayed front and centre on the profile page. The 'Find Friends' button links to a page where one can search for other users and send requests to connect with them. Once connected, friends are viewable on the app's dashboard or within the leaderboard, where everyone's scores can be seen and compared. Users can accept or reject connection requests. The user is notified through a red badge displayed on the profile icon in the navigation bar of pending connection requests. If there are outstanding requests, a button 'Friend Requests' is displayed in the user's profile which links to a page where users can accept or reject individual requests.

Rewards System

Social Tasker's reward system is an integral part of its aim to make users more productive. Its aim is to reward a user's consistence use and task completion. Social Taskers reward algorithm is inspired by an algorithm published by Frazier-Roberts and Peoples [2019]. Their algorithm gives good guidance on how to calculate a user's consistency and how to allocate points for completed tasks and was the main inspiration for Social Tasker's reward algorithm. Social Tasker's reward system takes the user's current daily and weekly streak into account and increases the calculated score if the user has remained consistent. Inversely, the score is reduced if the user has been inconsistent. The algorithm only applies this added bonus to the first task of the day. A higher score is achieved for completing tasks of a higher priority whilst it also aims to reward users that plan their tasks and allocate additional points for this. The algorithm's pseudocode can be found below.

```
// Calculate new Score  
// Initialize new variables  
newScore = 0  
newWeekStreak = user_week_streak or 0  
newDayStreak = user_day_streak or 0  
daysSinceLastTask = 0  
  
// Update daily streak  
newDayStreak = newDayStreak + 1  
  
// Update weekly streak  
if newDayStreak % 7 = 0 then  
    newDayStreak = 0  
    newWeekStreak = newWeekStreak + 1  
endif
```

// Task priority bonus

```
if taskData.priority = "high" then
    newScore = newScore + 25
elseif taskData.priority = "medium" then
    newScore = newScore + 15
elseif taskData.priority = "low" then
    newScore = newScore + 5
endif
```

// Calculate days since last task

```
if userData.last_task_completed_date then
    timeSinceLastTask = current date -
    userData.last_task_completed_date
    daysSinceLastTask = round(timeSinceLastTask / (1000 * 3600 *
24))
endif
```

// Daily Streak Bonus

```
if userData.task_day_streak > 0 and daysSinceLastTask = 1 then
    newScore = newScore + 20 * userData.task_day_streak
endif
```

// Weekly Streak Bonus

```
if userData.task_week_streak > 0 and daysSinceLastTask = 1 then
    newScore = newScore + 50 * userData.task_week_streak
endif
```

```

// Task on time completed bonus
dueDate = new Date(taskData.due_date)
completedDate = current date
timeDifference = dueDate - completedDate
if timeDifference > 0 then
    newScore = newScore + 50
endif

// Reward user for planning tasks ahead
createdDate = new Date(taskData.created_at)
planningDifference = (dueDate - createdDate) / (1000 * 60 * 60 * 24)
newScore = newScore + 2 * planningDifference

// Round the new score
newScore = round(newScore)

// Penalise for inconsistent task completion
if userData.last_task_completed_date then
    if daysSinceLastTask > 5 then
        newScore = newScore - 50
    elseif daysSinceLastTask > 1 then
        newScore = newScore - 25
    elseif daysSinceLastTask > 10 then
        newScore = newScore - 100
    endif
endif

// Update Neo4j and Firebase database with the new score.

```

Code Review

The project's monorepo can be found at this [link](#).

The codebase is split up between two main parts, the backend and frontend. The backend code consists of the API endpoints whilst the frontend is all the React Native code. The backend is written in JavaScript using ExpressJS to simplify the creation of API endpoints. ExpressJS allows for the implementation of middleware which are functions that have access to the request before executing the code within each endpoint. Two custom middlewares are used within Social Tasker, a data validator and an API key validator. The data validator ensures the information sent in the request body or parameters is valid and handles the relevant error in the case that it is not. Here is an example of the validator in charge of validating user data for use in creating or updating the users' table in the database.

```
7 // Validate user data
8 ~ const userValidator = [
9     body("email", "Field cannot be empty.").not().isEmpty(),
10    body("email", "Invalid email").isString().isEmail(),
11    body("name", "Field cannot be empty.").not().isEmpty(),
12    body("name", "Invalid name").isString(),
13    body("user_id", "Field cannot be empty.").not().isEmpty(),
14    body("user_id", "Invalid ID").isString(),
15 ];
```

Figure 12: Validator code for creating or updating the user.

Notice that the relevant error messages are defined here, making the errors very easy to handle on the front end.

The API key validator, incorporated into all endpoints, works similarly in that it checks whether the submitted key in the 'X-API-Key' header field is the correct one. It also handles errors as above. Incorporating an API key and rate limiter prevents unauthorized access and misuse of these endpoints. The rate limiter limits each IP address to 100 requests per 15-minute interval. ExpressJS makes it very easy to implement this throughout the whole application as seen in the code extract below.

```

36 // API Key Middleware
37 const apiKeyMiddleware = (req, res, next) => {
38   const apiKey = req.get('X-API-Key'); // Get API key from the request header
39   if (!apiKey) {
40     return res.status(401).send({ error: 'API key is required' });
41   }
42
43   // Check if API key is valid
44   if (apiKey !== process.env.CREATE_API_KEY) {
45     return res.status(403).send({ error: "Invalid API key" });
46   }
47
48   next(); // Proceed to the next middleware or route handler
49 };

```

Figure 13: API key validator code.

```

51 // Set up rate limiter
52 const limiter = rateLimit({
53   windowMs: 15 * 60 * 1000, // 15 minutes
54   limit: 100, // Limit each IP to 100 requests per `window` (here, per 15 minutes).
55   standardHeaders: "draft-7", // draft-6: `RateLimit-*` headers; draft-7: combined
    `RateLimit` header
56   legacyHeaders: false, // Disable the `X-RateLimit-*` headers.
57 });
58
59 // Apply rate limiter to all requests
60 app.use(limiter);

```

Figure 14: Rate limiter code.

Endpoints are identified and called according to standard REST endpoint grammar. Below is an example of an endpoint. This endpoint returns all the tasks of a user which is in turn used to display them within the user's dashboard or task list. In this example the GET request incorporates an API key validator in line 520 and an in-line validator for the parameter 'userId' passed in the request URL (lines 522-527).

```

518 // Get tasks of a user
519 app.get( "/api/tasks/user/:userId",
520     apiKeyMiddleware,
521     // Validate user ID param
522     param("userId")
523         .isString()
524         .withMessage("Invalid user ID. Must be a string.")
525         .not()
526         .isEmpty()
527         .withMessage("User ID cannot be empty."),
528     (req, res) => {
529         const { userId } = req.params;

```

Figure 15: Endpoint naming convention and middleware implementation example.

Endpoints are mainly responsible for creating, reading, updating and deleting data in the databases. To interact with Firebase, the Firebase Admin SDK, provided by Google, is used. The SDK handles authentication and provides intuitive methods to interact with the database. The code is clean and easy to read as seen in the extract below where a user's details are returned based on user ID.

```

257 // Get user doc from firestore
258 db.collection("users")
259     .doc(userId)
260     .get()
261     .then((doc) => {
262         if (!doc.exists) {
263             return res.status(404).send({ message: "User not found" });
264         }
265         const userData = doc.data();
266         return res.status(200).send(userData);
267     })
268     .catch((error) => {
269         // Handle error
270         const { code, message } = error;
271         return res.status(code).send({ error: message });
272     });

```

Figure 16: Firebase Admin SDK usage example.

To keep track of relationships between users, a graph database, Neo4J, is used. Creating and querying relationships is made simple by its custom query language Cypher. An example of a Cypher query can be seen below. This query language aims to visually

represent the data it returns as seen with this query which returns all users who are connected by a relationship 'friend'.

```
289 // Get friends from Neo4j database
290 session
291   .run(
292     `MATCH (u:User {user_id: $userId})
293     RETURN u AS user
294     UNION ALL
295     MATCH (u:User {user_id: $userId})-[:friend]->(f:User)
296     RETURN f AS user`,
297     { userId }
298   )
299   .then((result) => {
300     const friends = result.records.map(
301       (record) => record.get("user").properties
302     );
303     return res.status(200).send(friends);
304   })
305   .catch((error) => {
306     // Handle error
307     const { code, message } = error;
308     return res.status(code).send({ error: message });
309   });
310 });
```

Figure 17: Cypher query example.

The front end is built using React Native and designed according to its component philosophy. Components are reused as much as possible to make the codebase small and readable. The ability to embed components within one another was used to reuse the task- and friend-list components. These are used within the dashboard and their respective screens.

Component embedding and the ease with which React Native allows developers to share context throughout the application. An auth context was created and wrapped around the whole project and used for user authentication. This makes the user's credentials available throughout the application and the ability to validate a user easily anywhere. Below is the code that returns the context provided and makes the user information and methods

available throughout the application. A similar method was used to provide context about the device's connection status.

```
104 // Return the provider
105 // This provider wraps the entire app in App.js
106 return (
107   <AuthContext.Provider
108     value={{
109       currentUser,
110       logout,
111       deleteAccount,
112       resetPassword,
113       apiUrl,
114     }}>
115     {children}
116   </AuthContext.Provider>
117 );
118 };
119
120 // Hook to use auth in components
121 export const useAuth = () => useContext(AuthContext);
122
```

Figure 18: Auth context provider.

The authentication control is quite simple. Firebase Authentication is the underlying provider and passes the auth context into the above component. Once a current user is present the below navigator will route the user to the correct section of the app.

```

225 function MainNavigator() {
239   return (
240     <ConnectivityContext.Provider value={{ isConnected }}>
241       <NavigationContainer>
242         <Stack.Navigator>
243           {currentUser ? (
244             <Stack.Screen
245               name='ProtectedRoutes'
246               component={ProtectedRoutes}
247               options={{ headerShown: false }}
248             />
249           ) : (
250             <Stack.Screen
251               name='PublicRoutes'
252               component={PublicRoutes}
253               options={{ headerShown: false }}
254             />
255           )}
256         </Stack.Navigator>
257       </NavigationContainer>
258     </ConnectivityContext.Provider>
259   );
260 }

```

Figure 19: Auth check navigator.

Individual components are responsible for fetching the data to be displayed by that component. All API requests are made using JavaScript Fetch. This is a simple way of making requests and intuitively handling their response and potential errors. These requests are made within React's `useEffect` lifecycle hook which runs once when the component mounts. This hook also accepts a second argument, an array of variables. When a change in these variables is detected the hook will run again. Below is an example of a call to the API backend.

```

29  useEffect(() => {
30    // fetch user data from backend
31    fetch(`${apiUrl}/api/friends/${currentUser.uid}`, {
32      method: "GET",
33      headers: {
34        "Content-Type": "application/json",
35        "X-API-Key": process.env.EXPO_PUBLIC_CREATE_API_KEY,
36      },
37    })
38    .then((response) => {
39      return response.json();
40    })
41    .then((data) => {
42      setFriends(data);
43      setIsLoading(false);
44    })
45    .catch((error) => {
46      setSnackBarVisible(true);
47      setSnackBarMessage("Failed to get friends", error.message);
48      navigation.navigate("LoggedInRoutes", { screen: "Home" });
49      console.log(error);
50    });
51  }, []);

```

Figure 20: API request example.

Providing the user with feedback is also very important. Success or failure messages are an integral part of any application's user experience. This is achieved in multiple ways. A loading overlay is displayed when the app is in a loading state and a snack bar status is presented to the user when appropriate. As seen in the example above upon the return of an error a snack bar is set to be visible with the appropriate message advising the user of the error (lines 46-47).

Providing offline capabilities is important for mobile applications. Local storage synchronises tasks between the user's device and the cloud. For example, the code extract below checks whether a local copy exists if it does it presents that in the task list screen, if not it gets the tasks from the database. To ensure that local storage and the database remain in sync, local storage is modified alongside databases during modification, creation or deletion of a task. On initial loading of the app's dashboard, if tasks are not found in local storage then it is created from the database copy.

```

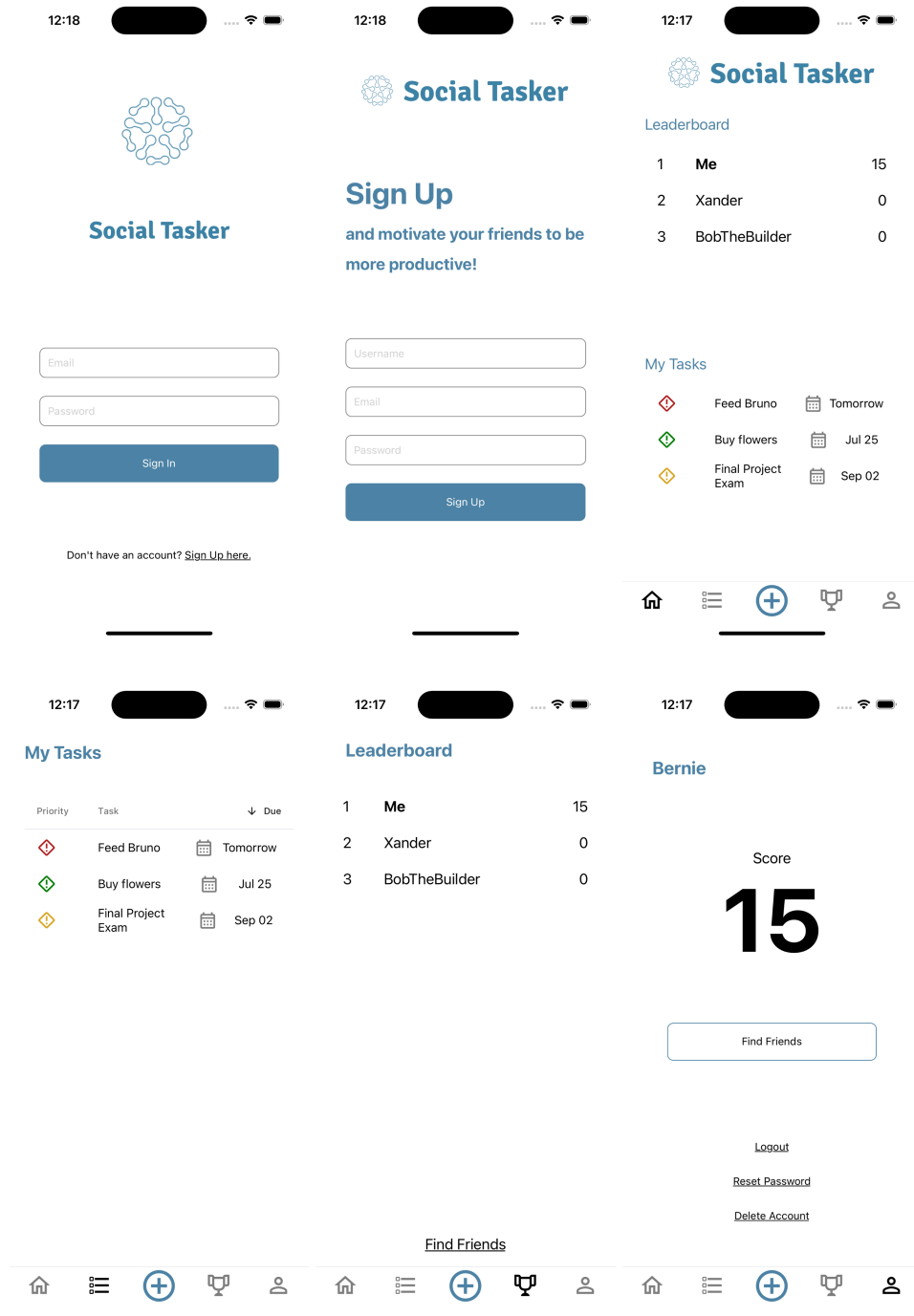
48  useEffect(() => {
49      // Get user favourites
50      // Check local storage first
51      AsyncStorage.getItem("favourites")
52      .then((value) => {
53          if (value) {
54              // Favourites found in local storage
55              setRecipeData(JSON.parse(value));
56              setIsLoading(false);
57          } else {
58              // Favourites not found in local storage
59              // Check firestore for user favourites
60              const userId = currentUser.uid;
61              const userRef = doc(db, "users", userId);
62              getDoc(userRef)
63              .then((docSnap) => {
64                  if (docSnap.exists()) {
65                      // User favourites found in firestore
66                      setIsLoading(false);
67                      setRecipeData(docSnap.data().favourites);
68                      // Save to local storage
69                      AsyncStorage.setItem(
70                          "favourites",
71                          JSON.stringify(docSnap.data().favourites)
72                      );
73                  } else {
74                      // User favourites not found in firestore
75                      setIsLoading(false);
76                  }
77              })
78              .catch((error) => {
79                  console.error("Error getting document:", error);
80                  setSnackBarVisible(true);
81                  setSnackBarMessage("Failed to get user data", error.message);
82              });
83          }
84      })
85  }, []);

```

Figure 21: Local storage example.

The Interface

Below are screenshots of all the screens of the final product.



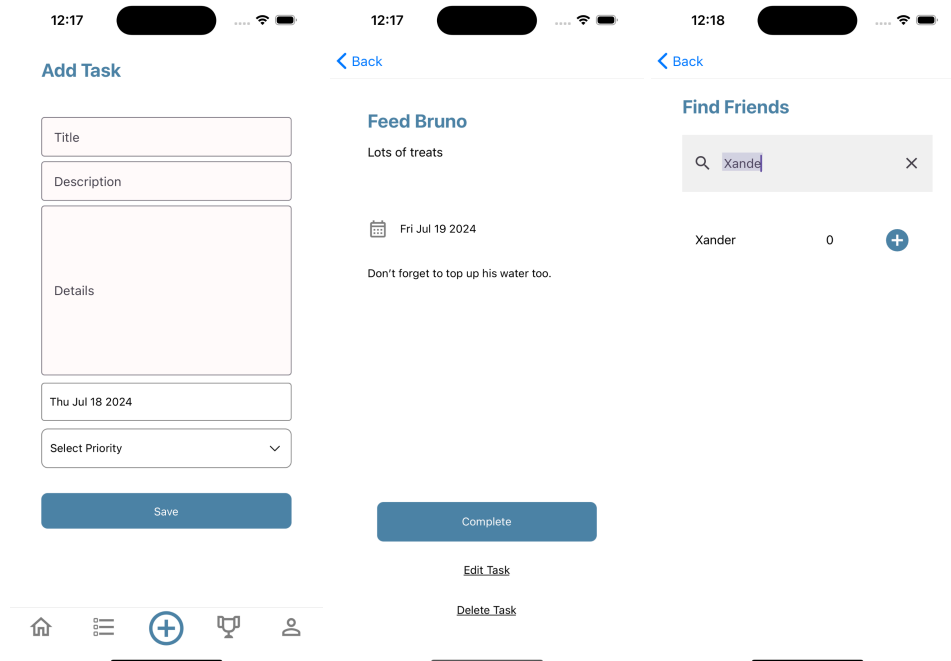


Figure 22: In app screenshots.

Evaluation

Testing and evaluation throughout the project's lifecycle is important. It ensures the product is aligned with what users need. Multiple rounds of testing supported the direction and integrity of Social Tasker. The project's evaluation plan consists of multiple stages and is described within this section.

Pre-development testing consisted of initial research into literature, architecture and literary works. The project's user experience and design were also assessed before development. During development, unit tests ensured that changes to code did not break functionality. The end product was tested through user integration testing, which evaluated the core functionality, performance and user experience of the product. Ongoing evaluation is important with any digital product to ensure users' evolving needs are met.

Testing

Initial research and testing revolved around determining key features for a task management application and how gamification affects users in general and specifically within the productivity space. As seen in the first chapter of this report, published works were reviewed to determine the effectiveness of gamification and how to implement it. Existing solutions were analysed to create an understanding of key features that are required. Research was also conducted as to the best way to tackle the social media aspect of Social Tasker from a technological perspective. To validate the viability of the technology stack that is intended to be used for this project a lightweight prototype was developed. The prototype demonstrated that using the Neo4J graph database was a viable option for Social Tasker's social media feature. It enables developers to easily create and query relationships between entities.

The application's design and user experience were tested early on in the prototype stage. This aims to receive feedback on the layout and design elements and evaluate whether any necessary features have been excluded. A simple questionnaire based on the System Usability Scale (SUS) was used to gather feedback. It was modified to allow the tester to give specific feedback about the design and features of the application. A score of 75 or higher was the goal before continuing with the project [12]. The average score for this SUS test was ~90, therefore no further changes were required. The SUS questionnaire can be found [here](#) and its results [here](#).

Unit tests were implemented for the API endpoints. These were written using Jest, a framework for writing unit tests in JavaScript. These tests test each endpoint and mainly check that endpoints do not allow unauthorised access, they return a successful response and they return the correct data. An example can be seen below for an endpoint that returns user data based on a search by name. The test first ensures no unauthorised access is allowed, then makes sure a response code of 200 is received and finally checks whether the returned data is the expected information. These unit tests can be very easily run by using

the command `npm test`. The tests were run whenever any changes were made to the API endpoints to ensure they worked correctly before deployment.

```
99 // GET search other users by name
100 describe("GET /api/search-friends/", () => {
101   it("Should not allow unauthorised access", async () => {
102     const res = await request(app).get(
103       `/api/search-friend/${friendData.name}`
104     );
105     expect(res.statusCode).toEqual(401);
106   });
107   it("Should return response 200", async () => {
108     const res = await request(app)
109       .get(`/api/search-friend/${friendData.name}`)
110       .set("x-api-key", API_KEY);
111     expect(res.statusCode).toEqual(200);
112   });
113   it("Should return the correct friend data", async () => {
114     const res = await request(app)
115       .get(`/api/search-friend/${friendData.name}`)
116       .set("x-api-key", API_KEY);
117     expect(res.body[0].name).toEqual(friendData.name);
118     expect(res.body[0].user_id).toEqual(friendData.user_id);
119     expect(res.body[0]).toHaveProperty("score");
120   });
121 });
```

Figure 23: API endpoint unit test example.

A peer review testing round was completed with the app at about 80% completion. This allowed potential users to get hands-on experience with Social Tasker and most importantly receive feedback from these testers. The feedback was invaluable mainly due to minor bugs being discovered. A tester commented: "It (Social Tasker) had a nice interface, and the signup process was mainly smooth until the app crashed upon finishing the signup; however, I still felt that the app had good error-catching, especially when not entering a valid email address." This error was caused by multiple queries being sent to the Neo4J database at once. To rectify this a second connection is established for the 'friend search' function to avoid this error appearing again. Another tester said: "The keyboard obstructs the password field while creating an account" and "The friend search is case sensitive which might cause an

inconvenience". This valuable feedback was immediately corrected by enabling a case-insensitive search and implementing a keyboard-avoiding view for the login and sign-up screens.

An end-user test was conducted after development. This test was comprised of users walking through all the features of Social Tasker and noting down whether or not the features worked. This type of integration testing is quick and efficient whilst getting the opinions of real-world users. During this final test users are also asked to note down any errors or bugs and suggest any improvements for future improvements. To access this questionnaire please follow [this link](#). Its results can be found [here](#).

The image shows a screenshot of a questionnaire used for user testing. It consists of four separate question cards, each with a title, a sub-instruction, and two radio button options: 'Yes' and 'No'. The cards are arranged vertically and have a light purple border. Each card also features a three-dot menu icon in the top right corner.

- Card 1:**
 - Title: I can create an account. *
 - Sub-instruction: (Please use your real email address, so that the password reset feature can be tested later on.)
 - Options: ☐ Yes, ☐ No
- Card 2:**
 - Title: I can search and add a friend. *
 - Sub-instruction: (Send the user 'Bernie' a friend request.)
 - Options: ☐ Yes, ☐ No
- Card 3:**
 - Title: I can log out of my account. *
 - Options: ☐ Yes, ☐ No
- Card 4:**
 - Title: I can log back into my account. *
 - Options: ☐ Yes, ☐ No

I can create a new task. *

☐ Yes

☐ No

I can edit a task. *

☐ Yes

☐ No

The new task appears on the dashboard and on the task list page. *

☐ Yes

☐ No

I can sort the task list by due date *

(Please create multiple tasks to test this)

☐ Yes

☐ No

When I complete a task, points are added to my profile. *

(Viewable on the profile page)

☐ Yes

☐ No

I can delete a task. *

(Please create a new one first)

☐ Yes

☐ No

I can reset my password. *

(A email will be sent to help you reset your password)

☐ Yes

☐ No

I can delete my account. *

☐ Yes

☐ No

I think this app will help me be more productive. *

☐ Yes

☐ Maybe

☐ No

Any suggestions are welcome.

Long answer text

Error details.

Long answer text

Figure 24: End user final test questions.

This integration test went well with no significant bugs or errors reported. Some new features were suggested and are discussed in the next section.

Critical Evaluation

The development phase of this project was a success taking into consideration the available resources and expertise. The product runs smoothly and includes all intended features. Feedback was well received and actions were taken based thereon to make sure the product meets the users' needs.

As with any project, there were many takeaways on how the next project can be done better. During the design and planning phase, it was underestimated how many varying user actions should lead to the same outcome for users to quickly get things done. For instance, adding a new friend would be too cumbersome to go to the profile page every time so a link was added on the leaderboard to quickly allow for quicker access to this feature. During the design phase, the user's onboarding experience was not really considered. In development, a small onboarding flow was created to have the user add friends immediately after signing up since this is the main feature of Social Tasker.

From a development perspective, some challenges also arose. Component props are passed to the 'edit task' screen to allow the user to make changes to a task. Initially, this screen was used for both creating and editing a task however the props seem to not be removed during navigation and removing them was a challenge. To avoid the previously edited task's details appearing in the form when creating a new task, two separate components were created. A bug was also discovered with the package *react-native-async-storage*'s method *mergeItem*. The details of this issue can be found [here](#). To circumvent this bug the entire local storage is copied, modified with new data and replaced.

A free version of Neo4J is used for this project. Although this is a generous and great feature, a drawback of this is that the database instance pauses after some time of inactivity and therefore won't receive any queries. To overcome this a simple script was created that runs a query on a cron job, running once a day to keep the database instance 'alive'. This

query simply asks for all nodes in the database. Overall these technical challenges were met and overcome.

The success of the overall project will be based on user engagement. The consistent use of the application over several months will indicate that users find it useful and enjoy using it. Several services are available to track these interactions. Task completion, bugs, user churn and so on are all valuable metrics that will help to improve and evaluate the success of Social Tasker. Metric-based goals will help quantify the success of the product. For example, having more than half the users complete tasks on more than 60% of the days would be considered a success. The preliminary results from the end user test show that users are not sure whether this app would be useful or not.

Ultimately the improvements to Social Tasker will be dictated by the users' evolving needs over time. More animations and user interactions could be added to enhance user engagement and enjoyment. For example, page transitions and state changes could be animated to make the application more interesting to use. Notifications could improve the app's gamification by notifying users when one of their friends has completed a task, potentially urging them to view their score or perhaps even complete a task themselves. Interface A/B testing could aid in choosing a better design according to users' preferences. Two different designs could be tested across two user groups and their engagement could be evaluated to determine the design with the better user experience.

Conclusion

This project has detailed the development of a novel social task management application called Social Tasker. Through extensive research into existing solutions, relevant literature, and enabling technologies, a comprehensive understanding of the problem space and potential design approaches was established. The integration of a social network and gamification can yield good results in terms of increased user productivity and user retention. Existing solutions are not utilising social networks to encourage users to be more productive. The iterative design process outlined the key features and functionality of Social Tasker, leveraging social mechanisms to enhance task planning and accountability among users. The detailed code review highlighted the technical implementation, showcasing the use of modern web frameworks, APIs, and databases to bring the application to life. Rigorous testing procedures were implemented to validate the usability, performance, and security of the Social Tasker platform. User feedback and a critical review of the application's strengths and limitations provided valuable insights to guide future development. In conclusion, Social Tasker represents a compelling social-centric approach to task management that addresses key pain points experienced by individuals and teams. The successful completion of this project demonstrates the viability of the proposed solution and lays the groundwork for further refinement and deployment to a wider user base. The learnings and methodologies applied throughout this endeavour can serve as a valuable reference for similar efforts in the realm of collaborative productivity tools.

Links

- Code repository: <https://github.com/bernardoosthuizen/uol-finalProject>
- Social Tasker (available on Expo Go):
<https://expo.dev/preview/update?message=Final%20v1.0&updateRuntimeVersion=1.0.0&createdAt=2024-08-13T02%3A50%3A50.099Z&slug=exp&projectId=45ee6063-a1db-4fb3-b17a-7cb73f337a92&group=843ce36b-3b67-4b5b-bb87-3b438f71e21c>



- Design SUS questionnaire: <https://forms.gle/AxwwVRUrrhsjkYTb8>
- Design SUS results:
<https://docs.google.com/spreadsheets/d/1S03i0HokxrZrzelX5GaTnsxAct7FD7T0dE6UAOwhtD4/edit?usp=sharing>
- Interim feedback questionnaire: <https://forms.gle/gPS16VBtzfNbAV2BA>
- Final end-user testing questionnaire: <https://forms.gle/NZQDfJQyQEuYdhG26>
- Final end-user test results:
<https://docs.google.com/spreadsheets/d/17Zqi6MEjiknoz2X4TFsE-CsK75DCwRfDEH5FG5qbhBg/edit?usp=sharing>

References

- [1] Alphabet Inc. 2024. Firebase Products. Firebase Products. <https://firebase.google.com/products-build>
- [2] Asana Inc. 2024. Asana Features. Asana's Work Tracking & Project Management Features. <https://asana.com/features>
- [3] Atlassian Corporation Plc. 2024. Agile Project Management. What is Agile? | Atlassian. <https://www.atlassian.com/agile>
- [4] Dennis L. Kappen, Jens Johannsmeier, and Lennart E. Nacke. Deconstructing 'gamified' task-management applications. In Proceedings of the First International Conference on Gameful Design, Research, and Applications (Gamification '13), (New York, NY, USA, 2013), Association for Computing Machinery, 139–142.
- [5] Doist Inc. 2024. Todoist Features. Features | Todoist. <https://todoist.com/features>
- [6] Figma Inc. 2024. Figma Features. Free Online Wireframe Tool For Teams | Figma. <https://www.figma.com/wireframe-tool/>
- [7] H.R. Vyawahare, P.P. Karde, and V.M. Thakare. A hybrid database approach using graph and relational database. In International Conference on Research in Intelligent and Computing in Engineering (RICE), (Universidad Don Bosco (UDB), El Salvador, 2018), IEEE, 1-4
- [8] Linear Orbit Inc. 2024. Features. Features – Linear. <https://linear.app/features>
- [9] Neo4j, Inc. 2024. Neo4j Graph Database. Native Graph Database | Neo4j Graph Database Platform. <https://neo4j.com/product/neo4j-graph-database/>

- [10] Soad Almagdy. 2018. Comparative Analysis of Relational and Graph Databases for Social Networks. International Journal of Soft Computing and Engineering (IJSCE), (April 2018). 2, 2, 509-512.
- [11] Steven Frazier-Roberts and Cathryn Peoples. 2019. The Design of a Gamification Algorithm in a Music Practice Application. Open Journal of Web Technologies (OJWT), 6, 1, 16-30.
- [12] Umair Rehman, Amir Zaib Abbasi, Ding Hooi Ting, Maria Hassan, and Nadine Khair. 2024. Exploring the impact of gamified experiences on user engagement in fitness apps: A GAMEFULQUEST perspective. IEEE Transactions on Engineering Management. 71, (January 2024), 3613–3628.
- [13] Wikimedia Foundation, Inc. 2024. System usability scale. System usability scale - Wikipedia. https://en.wikipedia.org/wiki/System_usability_scale